

```

In [1]: import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load dataset (auto-download)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

# Normalize data (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape for CNN (add channel dimension)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Class Labels
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Build CNN model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train model
history = model.fit(x_train, y_train, epochs=10,
                    validation_data=(x_test, y_test))

# Evaluate model
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)

# Plot accuracy
plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='val accuracy')
plt.legend()
plt.show()

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>

29515/29515 ————— 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>

26421880/26421880 ————— 5s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>

5148/5148 ————— 0s 1us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>

4422102/4422102 ————— 0s 0us/step

```
C:\Users\System21\AppData\Roaming\Python\Python311\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/10

1875/1875 ————— 12s 6ms/step - accuracy: 0.7554 - loss: 0.6837 - val_accuracy: 0.8772 - val_loss: 0.3463

Epoch 2/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.8804 - loss: 0.3281 - val_accuracy: 0.8835 - val_loss: 0.3185

Epoch 3/10

1875/1875 ————— 12s 6ms/step - accuracy: 0.8995 - loss: 0.2723 - val_accuracy: 0.8815 - val_loss: 0.3251

Epoch 4/10

1875/1875 ————— 13s 7ms/step - accuracy: 0.9104 - loss: 0.2404 - val_accuracy: 0.9013 - val_loss: 0.2764

Epoch 5/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.9211 - loss: 0.2136 - val_accuracy: 0.9020 - val_loss: 0.2622

Epoch 6/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.9290 - loss: 0.1916 - val_accuracy: 0.9016 - val_loss: 0.2748

Epoch 7/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.9364 - loss: 0.1721 - val_accuracy: 0.9084 - val_loss: 0.2590

Epoch 8/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.9407 - loss: 0.1566 - val_accuracy: 0.9095 - val_loss: 0.2774

Epoch 9/10

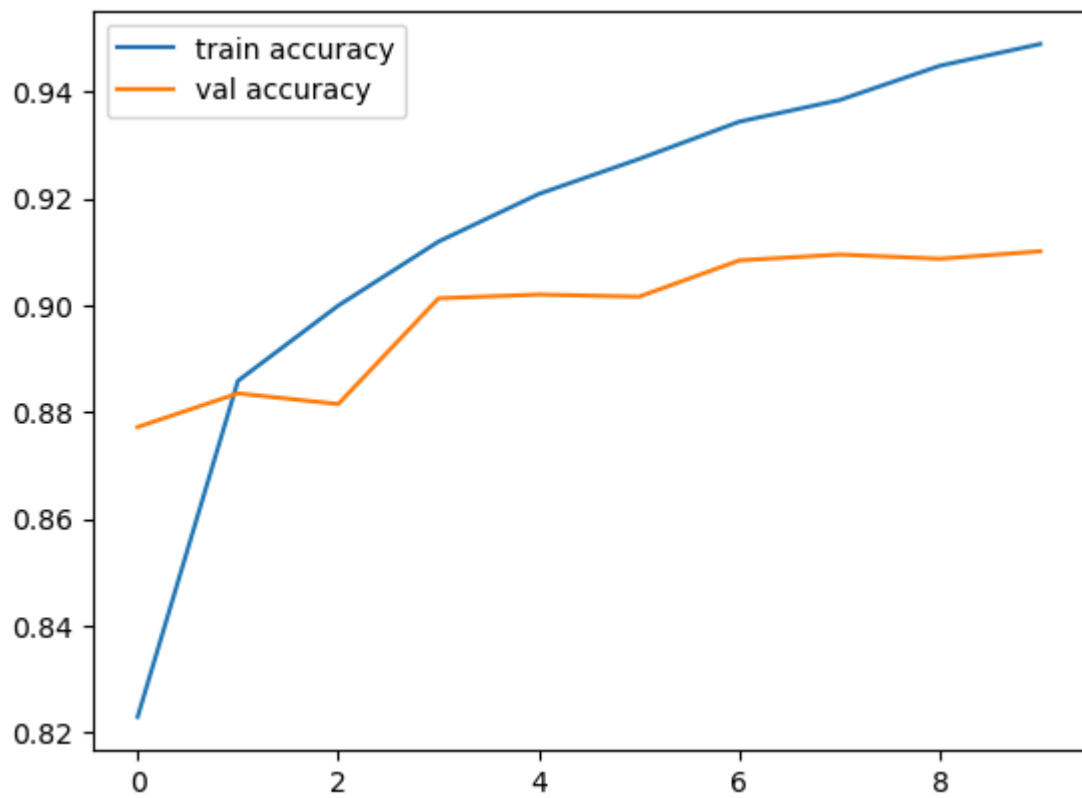
1875/1875 ————— 11s 6ms/step - accuracy: 0.9473 - loss: 0.1423 - val_accuracy: 0.9087 - val_loss: 0.2716

Epoch 10/10

1875/1875 ————— 11s 6ms/step - accuracy: 0.9506 - loss: 0.1302 - val_accuracy: 0.9101 - val_loss: 0.2870

313/313 ————— 1s 3ms/step - accuracy: 0.9067 - loss: 0.2995

Test Accuracy: 0.910099983215332



```
In [2]: # 📌 1. Show sample images
plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i].reshape(28,28), cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.suptitle("Sample Training Images")
plt.show()
```

Sample Training Images

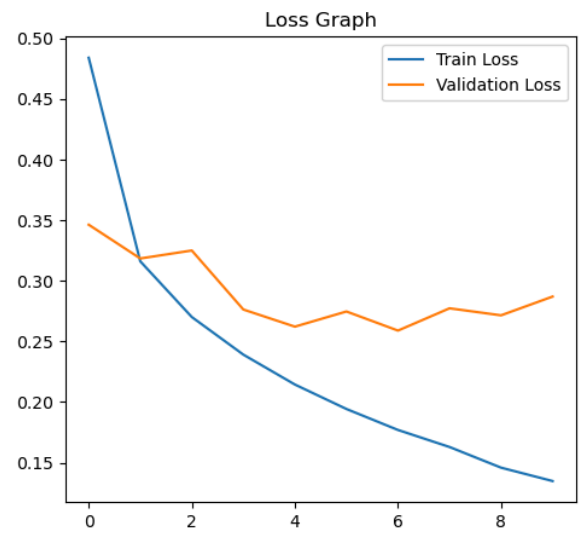
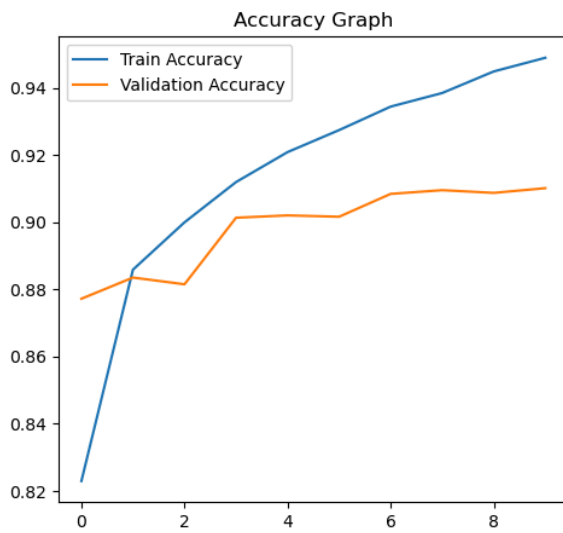


```
In [3]: plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Graph")

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Graph")

plt.show()
```



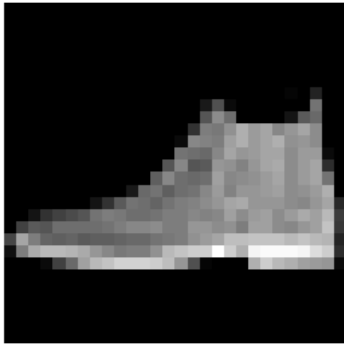
```
In [9]: # Generate predictions
predictions = model.predict(x_test)

# Convert probabilities → class labels
import numpy as np
pred_labels = np.argmax(predictions, axis=1)
plt.figure(figsize=(10,10))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(f"Pred: {class_names[pred_labels[i]]}\nActual: {class_names[y_test[i]]}")
    plt.axis('off')
plt.suptitle("Model Predictions")
plt.show()
```

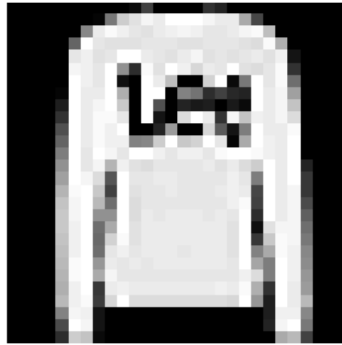
313/313 ————— 1s 2ms/step

Model Predictions

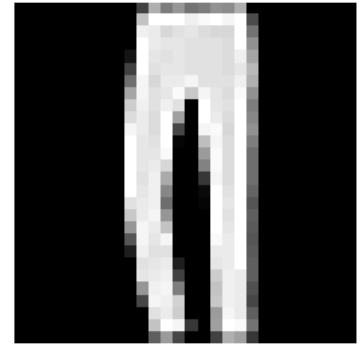
Pred: Ankle boot
Actual: Ankle boot



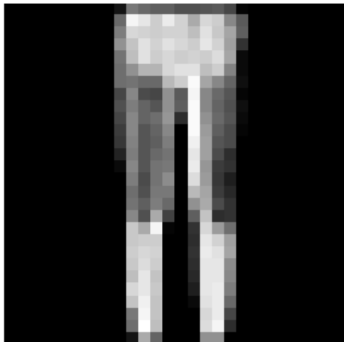
Pred: Pullover
Actual: Pullover



Pred: Trouser
Actual: Trouser



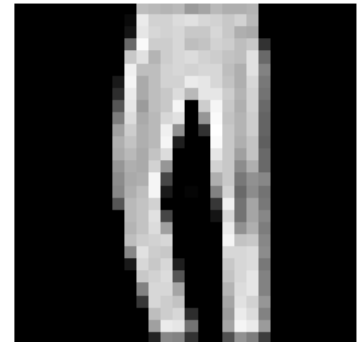
Pred: Trouser
Actual: Trouser



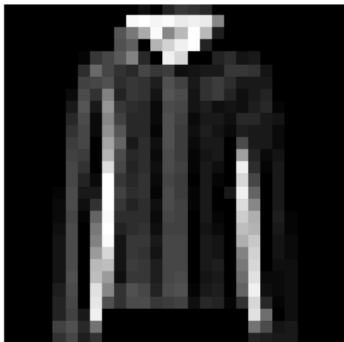
Pred: Shirt
Actual: Shirt



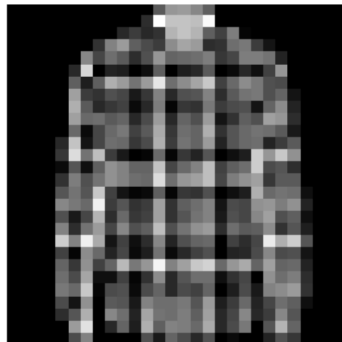
Pred: Trouser
Actual: Trouser



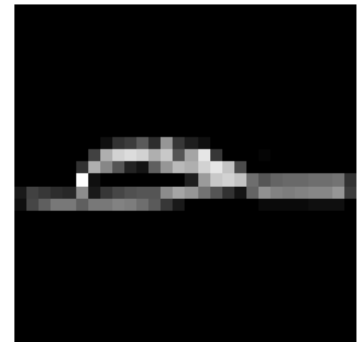
Pred: Coat
Actual: Coat



Pred: Coat
Actual: Shirt



Pred: Sandal
Actual: Sandal



In []:

In []: